



4

MUSICAL OVERTONES WITH KARPLUS-STRONG



One of the main characteristics of any musical sound is its pitch, or *frequency*. This is the sound's number of vibrations per second in hertz (Hz). For example, the fourth string of an acoustic guitar produces a D note with a frequency of 146.83 Hz. You can approximate this sound by creating a sine wave with a frequency of 146.83 Hz on a computer, as shown in **Figure 4-1**.

Unfortunately, if you play this sine wave on your computer, it won't sound anything like a guitar. Nor will it sound like a piano, or any other real-world musical instrument for that matter. What makes a computer sound so different from a musical instrument when playing the same note?

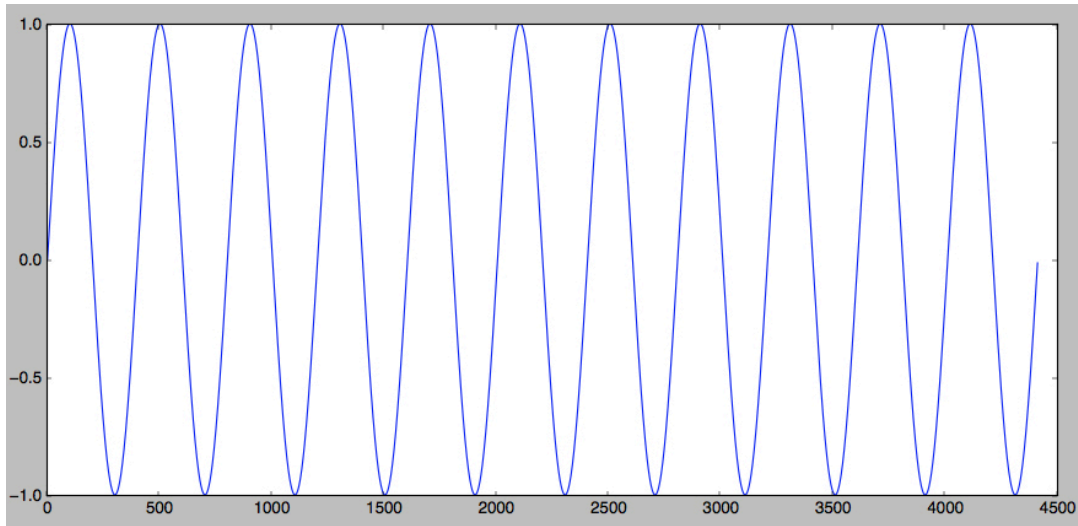


Figure 4-1: A sine wave at 146.83 Hz

When you pluck a string on a guitar, the instrument produces a mix of frequencies with varying intensity. The sound is most intense when the note is first struck, and the intensity dies off over time. In the case of plucking a guitar's D string, the dominant frequency you hear, called the *fundamental frequency*, is 146.83 Hz, but the sound also contains certain multiples of that frequency called *overtones*. In fact, the sound of any note on any instrument is composed of a fundamental frequency and overtones, and it's the combination of these different frequencies at different intensities that makes a guitar sound like a guitar, a piano sound like a piano, and so on. By contrast, a pure sine wave generated by a computer contains only a fundamental frequency, and no overtones.

You can see the evidence of overtones in *spectral plots*, like the one in **Figure 4-2** representing the D string of a guitar. A spectral plot shows all the frequencies present in a sound at a particular moment in time, as well as the intensity of those frequencies. Notice that there are many different peaks in the spectral plot shown in the figure, telling us that there are many frequencies present in the sound of the guitar's D string being plucked. Near the far left of the plot, the highest peak represents the fundamental frequency. The other peaks, representing

the overtones, are less intense, but they still contribute to the quality of the sound.

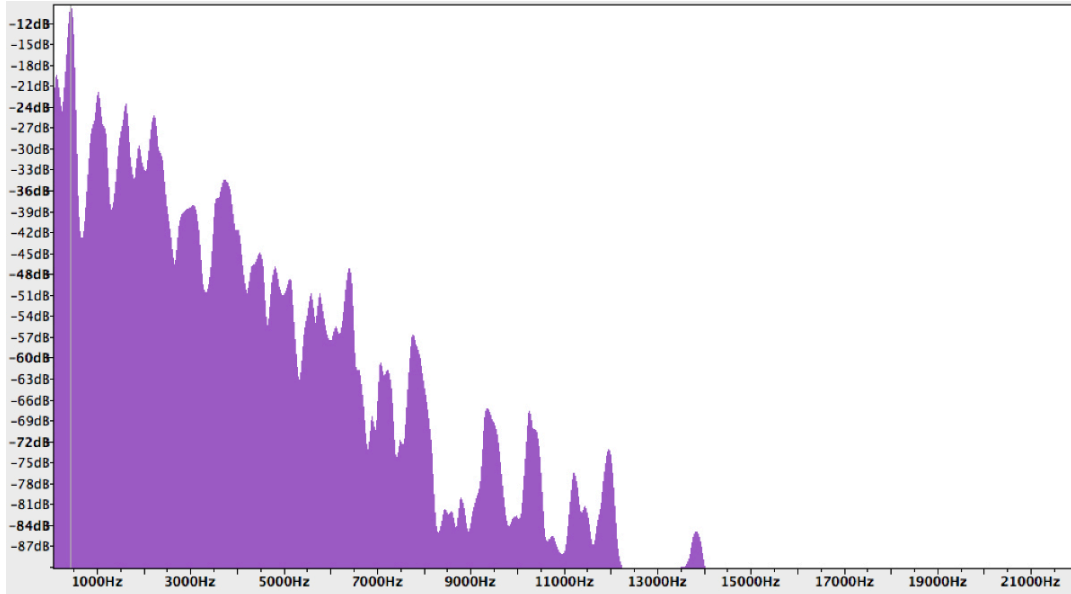


Figure 4-2: A spectral plot of the note D3 played on a guitar

As you can see, to simulate the sound of a plucked string instrument on the computer, you need to be able to generate both the fundamental frequency and the overtones. The trick is to use the Karplus-Strong algorithm. In this project, you'll generate five guitar-like notes of a musical scale (a series of related notes) using the Karplus-Strong algorithm. You'll visualize the algorithm used to generate these notes and save the sounds as WAV files. You'll also create a way to play them at random and learn how to do the following:

- Implement a ring buffer using the Python `deque` class.
- Use `numpy` arrays.
- Play WAV files using `pyaudio`.
- Plot a graph using `matplotlib`.
- Play the pentatonic musical scale.

How It Works

Imagine a string tied down at both ends, like a string on a guitar. When you pluck this string, it vibrates for a while, making a sound, and then settles back down to its resting position. At any given point in time while the string is vibrating, different parts of the string will be at different displacements from their resting position. These displacements can also be thought of as amplitudes of the sound wave produced by the vibrating string. The Karplus-Strong algorithm is a series of steps for generating and updating a series of these displacement or amplitude values to represent the motion of a wave along a plucked string. Play back those values as a WAV file and you get a pretty convincing simulation of a plucked string sound.

The Karplus-Strong algorithm stores displacement values in a *ring buffer* (also known as a *circular buffer*), a fixed-length buffer (just an array of values) that wraps around itself. In other words, when you reach the end of the buffer, the next element you access will be the first element in the buffer. (See [**“Implementing the Ring Buffer with deque”**](#) on [page 66](#) for more about ring buffers.)

The length (N) of the ring buffer is related to the fundamental frequency of the note you want to simulate according to the equation $N = S/f$, where S is the sampling rate (more on this later) and f is the frequency. At the start of the simulation, the buffer is filled with random values in the range $[-0.5, 0.5]$, which you might think of as representing the random displacement of a string when it's first plucked. As the simulation progresses, the values are updated according to the steps of the Karplus-Strong algorithm, which we'll outline next.

In addition to the ring buffer, you'll use a *samples buffer* to store the intensity of the sound at any particular time. This buffer represents the final sound data, and it's built up based on the values in the ring buffer. The length of the samples buffer and the sampling rate determine the length of the sound clip.

The Simulation

During each time step in the simulation, a value from the ring buffer is stored in the samples buffer, and then the values in the ring buffer are updated in a kind of feedback scheme, as shown in **Figure 4-3**. Once the samples buffer is full, you write its contents to a WAV file so the simulated note can be played back as audio. For each time step of the simulation, you follow these steps, which together make up the Karplus-Strong algorithm:

1. Store the first value from the ring buffer in the samples buffer.
2. Calculate the average of the first two elements in the ring buffer.
3. Multiply this average value by an attenuation factor (in this case, 0.995).
4. Append this value to the end of the ring buffer.
5. Remove the first element of the ring buffer.

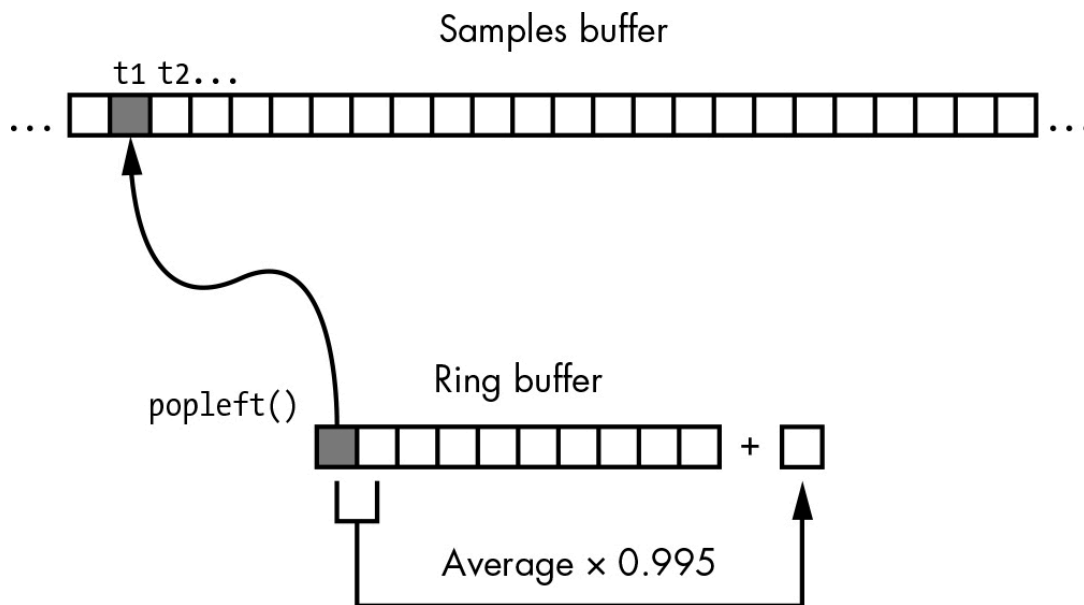


Figure 4-3: A ring buffer and the Karplus-Strong algorithm

This feedback scheme is designed to simulate a wave traveling through a vibrating string. The numbers in the ring buffer represent the energy of the wave at each point on the string. According to physics, the fundamental frequency of a vibrating string is inversely proportional to its length. Since we're interested in generating sounds of a certain frequency, we choose a ring buffer length inversely pro-

portional to the desired frequency (this is the $N = S/f$ formula mentioned earlier). The averaging that happens in step 2 of the algorithm acts as a *low-pass filter* that cuts off higher frequencies and allows lower frequencies through, thereby eliminating higher harmonics (that is, larger multiples of the fundamental frequency) because you're mainly interested in the fundamental frequency. The attenuation factor in step 3 simulates the loss of energy as the wave travels back and forth along the string. This corresponds to the fading of the sound over time.

The samples buffer that you add to in step 1 of the simulation represents the amplitude of the generated sound over time. Storing the attenuated values at the end of the ring buffer (step 4) and removing the first item from the ring buffer (step 5) ensures that a steady stream of gradually attenuating values will be passed to the samples buffer to build up the simulated sound.

Let's look at a simple example of the Karplus-Strong algorithm in action. **Table 4-1** represents a ring buffer at two consecutive time steps. Each value in the ring buffer represents the amplitude of the sound, which is the same as the displacement of a point on a plucked string from its rest position. The buffer has five elements, and they are initially filled with some numbers.

Table 4-1: A Ring Buffer at Two Time Steps in the Karplus-Strong Algorithm

Time step 1	0.1	-0.2	0.3	0.6	-0.5
Time step 2	-0.2	0.3	0.6	-0.5	-0.04975

As you go from time step 1 to time step 2, you apply the Karplus-Strong algorithm as follows. The first value in the first row, 0.1, is removed, and all subsequent values from time step 1 are added in the same order to the second row, which represents time step 2. The last value in

time step 2 is the attenuated average of the first and second values of time step 1, which is calculated as $0.995 \times ((0.1 + -0.2) \div 2) = -0.04975$.

The WAV File Format

The *Waveform Audio File Format (WAV)* is used to store audio data. This format is convenient for small audio projects because it's simple and doesn't require you to worry about complicated compression techniques.

In its simplest form, a WAV file consists of a series of values, where each value represents the amplitude of the stored sound at a given point in time. Each value is allotted a fixed number of bits, called the *resolution*. You'll use 16-bit resolution in this project. WAV files also have a set *sampling rate*, which is the number of times the audio is *sampled*, or read, every second. In this project, you use a sampling rate of 44,100 Hz, the rate used in audio CDs. In sum, when you generate a WAV file simulating the sound of a plucked string, it will contain 44,100 16-bit values for every second of audio.

For this project, you'll be using Python's `wave` module, which includes methods for working with WAV files. To get a feel for how it works, let's generate a five-second audio clip of a 220 Hz sine wave using Python. First you represent a sine wave using this formula:

$$A = \sin(2\pi ft)$$

Here, A is the amplitude of the wave, f is the frequency, and t is the current time index. Now you rewrite this equation as follows:

$$A = \sin(2\pi fi/R)$$

In this equation, i is the index of the sample, and R is the sampling rate. Using these two equations, you can create a five-second WAV file

for a 200 Hz sine wave as follows. (This code is available in *sine.py* in the chapter's GitHub repository.)

```
import numpy as np

import wave, math

sRate = 44100

nSamples = sRate * 5

❶ x = np.arange(nSamples)/float(sRate)

❷ vals = np.sin(2.0*math.pi*220.0*x)

❸ data = np.array(vals*32767, 'int16').tostring()

file = wave.open('sine220.wav', 'wb')

❹ file.setparams((1, 2, sRate, nSamples, 'NONE', 'uncompressed'))

❺ file.writeframes(data)

file.close()
```

You create a `numpy` array of numbers from `0` to `nSamples - 1` and divide those numbers by the sample rate to get the time value, in seconds, when each sample of the audio clip is taken ❶. This array represents the i/R portion of the sine wave equation discussed earlier. Next, you use the array to create a second `numpy` array, this one containing sine wave amplitude values, again following the sine wave equation ❷. The `numpy` array is a fast and convenient way to apply functions such as the `sin()` function to many values at once.

The computed sine wave values in the range $[-1, 1]$ are scaled to 16-bit values and converted to a string so they can be written to a WAV file

③. Then you set the parameters for the WAV file; in this case, it's a single-channel (mono), 2-byte (16-bit), uncompressed format ④. Finally, you write the data to the file ⑤. **Figure 4-4** shows the generated *sine220.wav* file in Audacity, a free audio editor. As expected, you see a sine wave of frequency 220 Hz, and when you play the file, you hear a 220 Hz tone for five seconds. (Note that you need to use the Zoom tool in Audacity to see the sine wave as shown in **Figure 4-4**.)

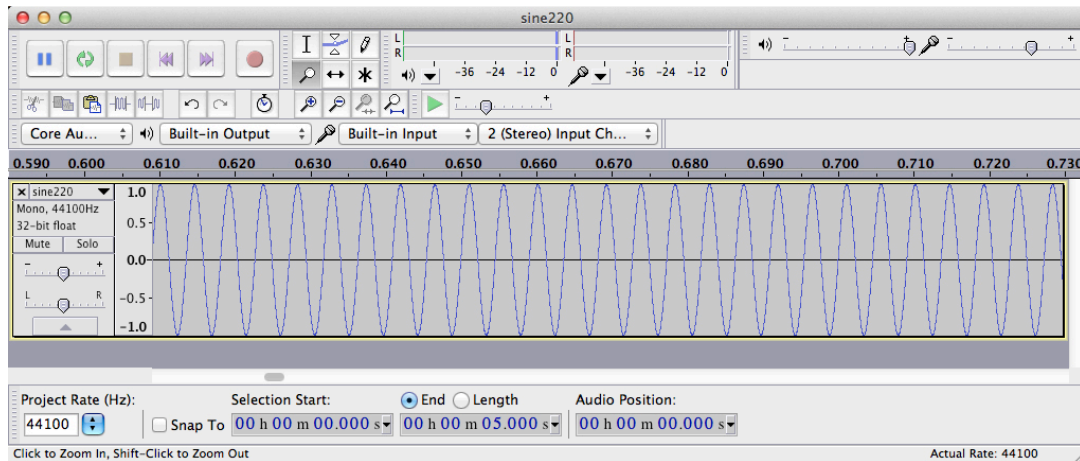


Figure 4-4: A sine wave at 220 Hz, zoomed in

In your project, once you've filled the samples buffer with audio data, you'll write it to a WAV file using the same pattern illustrated in **Figure 4-4**.

The Minor Pentatonic Scale

A *musical scale* is a series of notes in increasing or decreasing pitch (frequency). Often, all notes in a piece of music are chosen from a particular scale. A *musical interval* is the difference between two pitches. A *semitone* is a basic building block of a scale and is the smallest musical interval in Western music. A *tone* is twice the length of a semitone. The *major scale*, one of the most common musical scales, is defined by the interval pattern *tone-tone-semitone-tone-tone-tone-semitone*.

We'll briefly go into the pentatonic scale here, since you'll be generating musical notes in that scale. This section will explain the source of the frequency numbers used in the final program to generate notes with the Karplus-Strong algorithm. The *pentatonic scale* is a five-note musical scale. A variant of this scale is the *minor pentatonic scale*, which is defined by the interval pattern *(tone+semitone)-tone-tone-(tone+semitone)-tone*. Thus, the C minor pentatonic scale consists of the notes C, E-flat, F, G, and B-flat. **Table 4-2** lists the frequencies of the five notes of a C minor pentatonic scale that you'll generate using the Karplus-Strong algorithm. (Here, C4 designates C in the fourth octave of a piano, or *middle C*, by convention.)

Table 4-2: Notes in a Minor Pentatonic Scale

Note	Frequency (Hz)
C4	261.6
E-flat	311.1
F	349.2
G	392.0
B-flat	466.2

One aspect of this project will be stringing together random sequences of notes to create melodies. One of the reasons we're focusing on a minor pentatonic scale is that the notes of this scale sound pleasing no matter what order they're played in. Thus, the scale is particularly conducive to generating random melodies in a way that other scales, such as a major scale, are not.

Requirements

In this project, you'll use the Python `wave` module to create audio files in the WAV format. To implement the Karplus-Strong algorithm, you'll use the `deque` class from the Python `collections` module as a ring buffer and a `numpy` array as a samples buffer. You'll also use `matplotlib` to visualize the simulated guitar string, and you'll play back the WAV files with the `pyaudio` module.

The Code

Now let's develop the various pieces of code required to implement the Karplus-Strong algorithm and then put them together for the complete program. To see the full project code, skip ahead to **[“The Complete Code”](#)** on **[page 74](#)**. You can also download the code from the book's GitHub repository at **<https://github.com/mkvenkit/pp2e/tree/main/karplus>**.

Implementing the Ring Buffer with deque

Recall from earlier that the Karplus-Strong algorithm uses a ring buffer to generate a musical note. You'll implement the ring buffer using a `deque` container (pronounced “deck”), which is part of Python's `collections` module of specialized container data types. You can insert and remove elements from the beginning (head) or end (tail) of a `deque` (see **[Figure 4-5](#)**). This insertion and removal process is a $O(1)$, or a “constant time” operation, which means it takes the same amount of time regardless of how big the `deque` container gets.

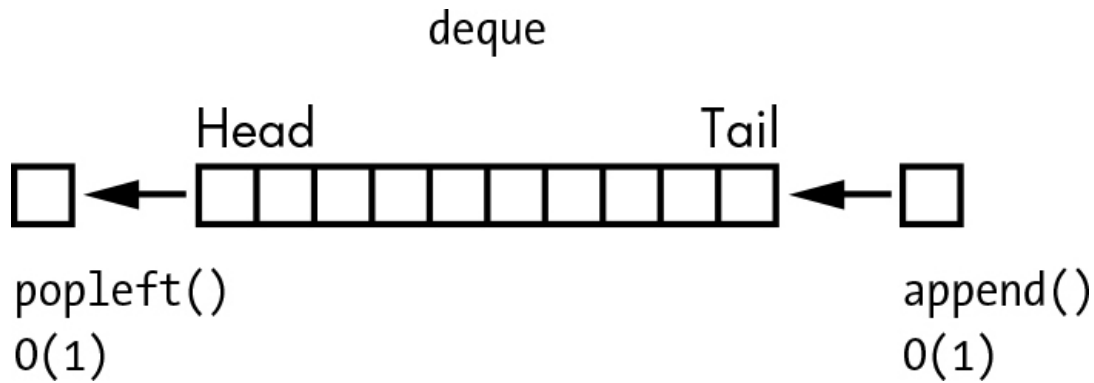


Figure 4-5: A ring buffer implemented using `deque`

The following code shows an example of how you would use `deque` in Python:

```
>>> from collections import deque

❶ >>> d = deque(range(10), maxlen=10)

>>> print(d)

deque([0, 1, 2, 3, 4, 5, 6, 7, 8, 9], maxlen=10)

❷ >>> d.append(10)

>>> print(d)

deque([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], maxlen=10)
```

You create the `deque` container by passing in a list created with the `range()` function ❶. You also specify the maximum length `maxlen` of the `deque` as `10`. Next, you append the element `10` to the end of the `deque` container ❷. When you then print the `deque`, you can see that `10` has been appended to the end of the `deque`, while the first element, `0`, has automatically been removed to maintain the `deque` container's maximum length of 10 elements. This scheme will allow you to simultaneously implement steps 4 and 5 of the Karplus-Strong

algorithm—adding a new value at the end of the ring buffer while removing the first value.

Implementing the Karplus-Strong Algorithm

You'll now implement the Karplus-Strong algorithm in the `generateNote()` function, using a `deque` container to implement the ring buffer and a `numpy` array to implement the samples buffer. In the same function, you'll also visualize the algorithm using `matplotlib`. The plot will show how the amplitudes of the plucked string change over time, in effect showing how the string moves as it vibrates.

You begin with some setup:

```
# initialize plotting
```

```
❶ fig, ax = plt.subplots(1)
```

```
❷ line, = ax.plot([], [])
```

```
def generateNote(freq):
```

```
    """generate note using Karplus-Strong algorithm"""
```

```
    nSamples = 44100
```

```
    sampleRate = 44100
```

```
❸ N = int(sampleRate/freq)
```

```
❹ if gShowPlot:
```

```
    # set axis
```

```
    ax.set_xlim([0, N])
```

```
ax.set_ylim([-1.0, 1.0])
```

```
line.set_xdata(np.arange(0, N))
```

```
# initialize ring buffer
```

```
⑤ buf = deque([random.random() - 0.5 for i in range(N)], maxlen=N)
```

```
# init samples buffer
```

```
⑥ samples = np.array([0]*nSamples, 'float32')
```

First you create a `matplotlib` figure ❶ and a line plot ❷, which you'll fill with data. Then you begin the `generateNote()` function definition, which takes the frequency of the note to be generated as a parameter. You set the number of samples in the sound clip and the sample rate to both be 44,100, which means the resulting clip will be one second long. Then you divide the sample rate by the desired frequency to set the length `N` of the Karplus-Strong ring buffer ❸. If the `gShow-Plot` flag is set ❹, you initialize the x and y range of the plot and initialize the x values to `[0, ... N-1]` using the `arange()` function.

You next initialize the ring buffer as a `deque` container with random numbers in the range `[-0.5, 0.5]`, setting the maximum length of the `deque` to `N` ❺. You also initialize the samples buffer as a `numpy` array of floats ❻. You set the length of the array to be the number of samples the sound clip will contain.

Next comes the heart of the `generateNote()` function, where you implement the steps of the Karplus-Strong algorithm and create the visualization:

```
for i in range(nSamples):
```

```
    ❶ samples[i] = buf[0]
```

❷ `avg = 0.995*0.5*(buf[0] + buf[1])`

❸ `buf.append(avg)`

`# plot of flag set`

❹ `if gShowPlot:`

`if i % 1000 == 0:`

`line.set_ydata(buf)`

`fig.canvas.draw()`

`fig.canvas.flush_events()`

`# samples to 16-bit to string`

`# max value is 32767 for 16-bit`

❺ `samples = np.array(samples * 32767, 'int16')`

❻ `return samples.tobytes()`

Here you iterate over each element in the samples buffer and carry out the steps of the Karplus-Strong algorithm. With each iteration, you copy the first element in the ring buffer to the samples buffer ❶. Then you perform the low-pass filtering and attenuation by averaging the first two elements in the ring buffer and multiplying the result by 0.995 ❷. This attenuated value is appended to the end of the ring buffer ❸. Since the `deque` representing the ring buffer has a maximum length, the `append()` operation also removes the first element from the buffer.

The `samples` array is converted into a 16-bit format by multiplying each value by 32,767 ❺ (a 16-bit signed integer can take values only

from $-32,768$ to $32,767$, and $0.5 \times 65,534 = 32,767$). Then the array is converted to a byte representation for the `wave` module, which you'll use to save this data to a file ⑥.

As the algorithm is running, you visualize how the ring buffer evolves ④. For every thousand samples, you update the `matplotlib` graph with the values in the ring buffer, and this shows how the data changes with time.

Writing a WAV File

Once you have the audio data, you can write it to a WAV file using the Python `wave` module. Define a `writeWAVE()` function to carry this out:

```
def writeWAVE(fname, data):
```

```
    # open file
```

```
    ① file = wave.open(fname, 'wb')
```

```
    # WAV file parameters
```

```
    nChannels = 1
```

```
    sampleWidth = 2
```

```
    frameRate = 44100
```

```
    nFrames = 44100
```

```
    # set parameters
```

```
    ② file.setparams((nChannels, sampleWidth, frameRate, nFrames,  
                     'NONE', 'noncompressed'))
```



```
❸ file.writeframes(data)
```

```
file.close()
```

You create a WAV file ❶ and set its parameters using a single-channel, 16-bit, noncompressed format ❷. Then you write the data to the file ❸.

Playing WAV Files with pyaudio

Now you'll use the Python `pyaudio` module to play the WAV files generated by the algorithm. `pyaudio` is a high-performance, low-level library that gives you access to sound devices on a computer. For convenience, you encapsulate the code in a `NotePlayer` class, as shown here:

```
class NotePlayer:
```

```
    # constructor
```

```
    def __init__(self):
```

```
        # init pyaudio
```

```
        ❶ self.pa = pyaudio.PyAudio()
```

```
        # open stream
```

```
        ❷ self.stream = self.pa.open(
```

```
            format=pyaudio.paInt16,
```

```
            channels=1,
```

```
            rate=44100,
```

```
            output=True)
```

```
# dictionary of notes
```

```
❸ self.notes = []
```

In the `NotePlayer` class's constructor, you first create the `PyAudio` object that you'll use to play the WAV file ❶. Then you open a 16-bit single-channel `PyAudio` output stream ❷. You also create an empty list that you'll later fill with filenames of the five pentatonic note WAV files ❸.

In Python, when all references to an object have been deleted, the object is destroyed by a process called *garbage collection*. At that time, the object's `__del__()` method, also known as a *destructor*, is called, if one is defined. Here's the `NotePlayer` class's destructor:

```
def __del__(self):  
  
    # destructor  
  
    self.stream.stop_stream()  
  
    self.stream.close()  
  
    self.pa.terminate()
```

This method ensures that the `PyAudio` stream is cleaned up when the `NotePlayer` object is destroyed. Failing to provide a `__del__()` method for a class can cause problems when objects are repeatedly created and destroyed, since some system-wide resources (like `pyaudio` in this case) may not be cleaned up properly.

The remaining methods of the `NotePlayer` class are devoted to building up a list of possible notes and playing them. First, here's the `add()` method, which is used to add a WAV filename to the class:

```
def add(self, fileName):
```

```
self.notes.append(fileName)
```

The method takes a filename corresponding to one of the generated WAV files as a parameter and adds it to the `notes` list you initialized in the class's constructor. The class will draw on this list when it wants to play a WAV file.

Next, let's look at the `play()` method used to play a note:

```
def play(self, fileName):
```

```
    try:
```

```
        print("playing " + fileName)
```

```
        # open WAV file
```

```
        ❶ wf = wave.open(fileName, 'rb')
```

```
        # read a chunk
```

```
        ❷ data = wf.readframes(CHUNK)
```

```
        # read rest
```

```
        while data != b":
```

```
            ❸ self.stream.write(data)
```

```
            ❹ data = wf.readframes(CHUNK)
```

```
        # clean up
```

```
        ❺ wf.close()
```

```
    except BaseException as err:
```

```
⑥ print(f"Exception! {err=}, {type(err)=}.\nExiting.")
```

```
exit(0)
```

Here you open the desired WAV file using Python's `wave` module ①. Then you read `CHUNK` frames (defined globally as 1,024 in this case) from the file into `data` ②. Next, within a `while` loop, you write the contents of `data` to the `PyAudio` output stream ③ and read the next chunk of data from the WAV file ④. Writing to the output stream has the effect of playing the audio through the default audio device of your computer, which is typically a speaker. You read the data in chunks to maintain the sample rate at the output side. If the chunks are too large and you take too much time in between reading and writing, the audio won't sound right.

The `while` loop continues for as long as there's more data to read—that is, until `data` is empty. At that point, you close the WAV file object ⑤. You handle any exceptions that may happen during the playback process (for example, the user pressing CTRL-C) by printing the error ⑥ and exiting the program.

Finally, the `NotePlayer` class's `playRandom()` method picks a random note from the five notes you've generated and plays it:

```
def playRandom(self):
```

```
    """play a random note"""
```

```
    index = random.randint(0, len(self.notes)-1)
```

```
    note = self.notes[index]
```

```
    self.play(note)
```

The method selects a random WAV filename from the `notes` list and passes that filename to the `play()` method to be played.

Creating Notes and Parsing Arguments

Now let's look at the program's `main()` function, which creates the notes and handles various command line options to play the notes:

```
def main():
```

```
-- snip --
```

```
    parser = argparse.ArgumentParser(description="Generating sounds  
with
```

```
                                Karplus-Strong Algorithm")
```

```
    # add arguments
```

```
    parser.add_argument('--display', action='store_true', required=False)
```

```
    parser.add_argument('--play', action='store_true', required=False)
```

```
    args = parser.parse_args()
```

```
    # show plot if flag set
```

```
    ❶ if args.display:
```

```
        gShowPlot = True
```

```
        plt.show(block=False)
```

```
    # create note player
```

```
    ❷ nplayer = NotePlayer()
```

```
    print('creating notes...')
```

```
    for name, freq in list(pmNotes.items()):
```

```
fileName = name + '.wav'
```

③ if not os.path.exists(fileName) or args.display:

```
    data = generateNote(freq)
```

```
    print('creating ' + fileName + '...')
```

```
    writeWAVE(fileName, data)
```

```
else:
```

```
    print('fileName already created. skipping...')
```

```
# add note to player
```

④ nplayer.add(name + '.wav')

```
# play note if display flag set
```

```
if args.display:
```

⑤ nplayer.play(name + '.wav')

```
    time.sleep(0.5)
```

```
# play a random tune
```

```
if args.play:
```

```
    while True:
```

```
        try:
```

⑥ nplayer.playRandom()

```
            # rest - 1 to 8 beats
```

```

❷ rest = np.random.choice([1, 2, 4, 8], 1,
                             p=[0.15, 0.7, 0.1, 0.05])

```

```

time.sleep(0.25*rest[0])

```

```

except KeyboardInterrupt:

```

```

    exit()

```

First you set up some command line options for the program using `argparse`, as discussed in earlier projects. The `--display` option will play each of the five notes in turn, while visualizing each note's waveform using `matplotlib`. The `--play` option generates a random melody using the five notes.

If the `--display` command line option was used ❶, you set up a `matplotlib` plot to show how the waveform evolves during the Karplus-Strong algorithm. The `plt.show(block=False)` call ensures that the `matplotlib` display method doesn't block. This way, when you call this function, it will return immediately and go on to the next statement. This is the behavior you need, since you're manually updating the plot every frame.

You next create an instance of the `NotePlayer` class ❷. Then you generate WAV files of the five notes in the C minor pentatonic scale. The frequencies for the notes are defined in the global dictionary `pmNotes`, which looks like this:

```

pmNotes = {'C4': 262, 'Eb': 311, 'F': 349, 'G': 391, 'Bb': 466}

```

To generate the notes, you iterate through the dictionary, first constructing a filename for the note using the dictionary key plus the `.wav` extension—for example, `C4.wav`. You use the `os.path.exists()` method to see whether the WAV file for a particular note has been created ❸. If so, you skip the computation for that note. (This is a handy

optimization if you're running this program several times.) Otherwise, you generate the note using the `generateNote()` and `writeWAVE()` functions you defined earlier. Once the note is computed and the WAV file created, you add the note's filename to the `NotePlayer` object's list of notes ④, and then you play the note if the `--display` command line option is used ⑤.

If the `--play` option is used, the `playRandom()` method in `NotePlayer` repeatedly plays a note at random from the five notes ⑥. For a note sequence to sound even remotely musical, you need to add rests between the notes played, so you use the `random.choice()` method from `numpy` to choose a random rest interval ⑦. This method also lets you choose the probability of the rest interval, which you set so that a two-beat rest is the most probable and an eight-beat rest the least probable. Try changing these values to create your own style of random music!

Running the Plucked String Simulation

To run the code for this project, enter this in a command shell:

```
$ python ks.py --display
```

As you can see in **Figure 4-6**, the `matplotlib` plot shows how the Karplus-Strong algorithm converts the initial random displacements to create waves of the desired frequency.

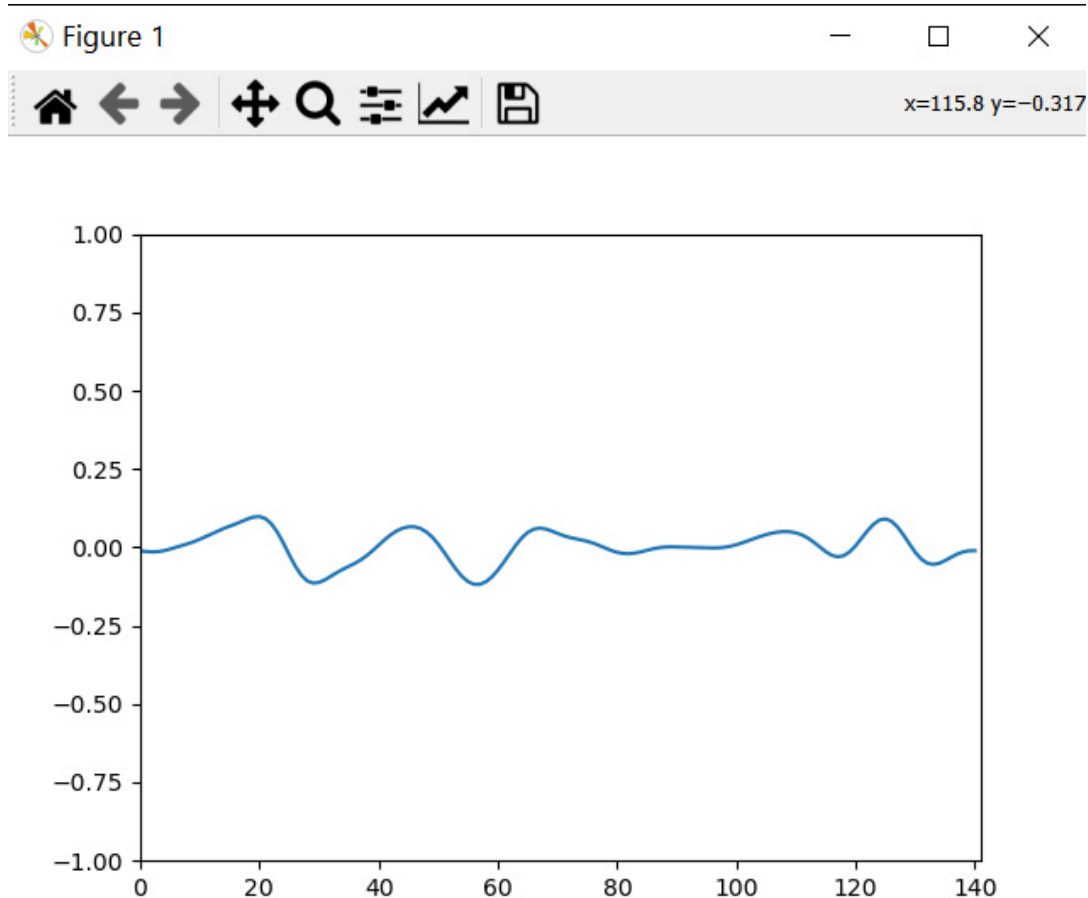


Figure 4-6: A sample run of the plucked string simulation

Now try playing a random sequence of notes using this command:

```
$ python ks.py --play
```

This should play a random note sequence using the generated WAV files of the pentatonic musical scale.

Summary

In this project, you used the Karplus-Strong algorithm to simulate the sound of plucked strings and played notes from generated WAV files. You learned how to implement the Karplus-Strong algorithm using a `deque` container as a ring buffer. You also learned about the WAV file format and how to play WAV files using `pyaudio`, as well as how to

use `matplotlib` to visualize a vibrating string. You even learned about the pentatonic musical scale!

Experiments!

Here are some ideas for experiments:

1. I've stated that the Karplus-Strong algorithm creates realistic plucked string sounds by generating overtones as well as the fundamental frequency of the note. But how do you know it's working? By creating spectral plots of your WAV files, like the one in [Figure 4-2](#). You can use the free program Audacity to do this. Open one of the WAV files in Audacity, and select **Analyze-Plot Spectrum**. You should see that the sound contains many frequencies.
2. Use the techniques you learned in this chapter to create a method that replicates the sound of two strings of different frequencies vibrating together. Remember, the Karplus-Strong algorithm produces a ring buffer full of sound amplitude values. You can combine two sounds by adding their amplitudes together.
3. Replicate the sound of two strings vibrating together, as described in the previous experiment, but add a time delay between the first and second string plucks.
4. Write a method to read music from a text file and generate musical notes. Then play the music using these notes. You can use a format where the note names are followed by integer rest time intervals, like this: C4 1 F4 2 G4 1 . . .

The Complete Code

Here's the complete code for this project:

"""

ks.py

Uses the Karplus-Strong algorithm to generate musical notes

in a pentatonic scale.

Author: Mahesh Venkitachalam

```
"""
```

```
import sys, os
```

```
import time, random
```

```
import wave, argparse
```

```
import numpy as np
```

```
from collections import deque
```

```
import matplotlib
```

```
# to fix graph display issues on macOS
```

```
matplotlib.use('TkAgg')
```

```
from matplotlib import pyplot as plt
```

```
import pyaudio
```

```
# show plot of algorithm in action?
```

```
gShowPlot = False
```

```
# notes of a pentatonic minor scale
```

```
# piano C4-E(b)-F-G-B(b)-C5
```

```
pmNotes = {'C4': 262, 'Eb': 311, 'F': 349, 'G':391, 'Bb':466}
```

```
CHUNK = 1024
```

```
# initialize plotting
```

```
fig, ax = plt.subplots(1)
```

```
line, = ax.plot([], [])
```

```
# write out WAV file
```

```
def writeWAVE(fname, data):
```

```
    """write data to WAV file"""
```

```
    # open file
```

```
    file = wave.open(fname, 'wb')
```

```
    # WAV file parameters
```

```
    nChannels = 1
```

```
    sampleWidth = 2
```

```
    frameRate = 44100
```

```
    nFrames = 44100
```

```
    # set parameters
```

```
    file.setparams((nChannels, sampleWidth, frameRate, nFrames,
```

```
                    'NONE', 'noncompressed'))
```

```
    file.writeframes(data)
```

```
    file.close()
```

```
def generateNote(freq):

    """generate note using Karplus-Strong algorithm"""

    nSamples = 44100

    sampleRate = 44100

    N = int(sampleRate/freq)

    if gShowPlot:

        # set axis

        ax.set_xlim([0, N])

        ax.set_ylim([-1.0, 1.0])

        line.set_xdata(np.arange(0, N))

    # initialize ring buffer

    buf = deque([random.random() - 0.5 for i in range(N)], maxlen=N)

    # init sample buffer

    samples = np.array([0]*nSamples, 'float32')

    for i in range(nSamples):

        samples[i] = buf[0]

        avg = 0.995*0.5*(buf[0] + buf[1])

        buf.append(avg)

    # plot of flag set
```

```
if gShowPlot:

    if i % 1000 == 0:

        line.set_ydata(buf)

        fig.canvas.draw()

        fig.canvas.flush_events()

# samples to 16-bit to string

# max value is 32767 for 16-bit

samples = np.array(samples * 32767, 'int16')

return samples.tobytes()

# play a WAV file

class NotePlayer:

    # constructor

    def __init__(self):

        # init pyaudio

        self.pa = pyaudio.PyAudio()

        # open stream

        self.stream = self.pa.open(

            format=pyaudio.paInt16,

            channels=1,
```

```
        rate=44100,

        output=True)

# dictionary of notes

self.notes = []

def __del__(self):

    # destructor

    self.stream.stop_stream()

    self.stream.close()

    self.pa.terminate()

# add a note

def add(self, fileName):

    self.notes.append(fileName)

# play a note

def play(self, fileName):

    try:

        print("playing " + fileName)

        # open WAV file

        wf = wave.open(fileName, 'rb')

        # read a chunk
```

```
data = wf.readframes(CHUNK)

# read rest

while data != b"":

    self.stream.write(data)

    data = wf.readframes(CHUNK)

# clean up

wf.close()

except BaseException as err:

    print(f"Exception! {err=}, {type(err)=}.\nExiting.")

    exit(0)

def playRandom(self):

    """play a random note"""

    index = random.randint(0, len(self.notes)-1)

    note = self.notes[index]

    self.play(note)

# main() function

def main():

    # declare global var

    global gShowPlot
```



```
parser = argparse.ArgumentParser(description="Generating sounds  
with
```

```
    Karplus-Strong Algorithm.")
```

```
# add arguments
```

```
parser.add_argument('--display', action='store_true', required=False)
```

```
parser.add_argument('--play', action='store_true', required=False)
```

```
args = parser.parse_args()
```

```
# show plot if flag set
```

```
if args.display:
```

```
    gShowPlot = True
```

```
    # plt.ion()
```

```
    plt.show(block=False)
```

```
# create note player
```

```
nplayer = NotePlayer()
```

```
print('creating notes...')
```

```
for name, freq in list(pmNotes.items()):
```

```
    fileName = name + '.wav'
```

```
    if not os.path.exists(fileName) or args.display:
```

```
        data = generateNote(freq)
```

```
print('creating ' + fileName + '...')

writeWAVE(fileName, data)

else:

    print('fileName already created. skipping...')

# add note to player

nplayer.add(name + '.wav')

# play note if display flag set

if args.display:

    nplayer.play(name + '.wav')

    time.sleep(0.5)

# play a random tune

if args.play:

    while True:

        try:

            nplayer.playRandom()

            # rest - 1 to 8 beats

            rest = np.random.choice([1, 2, 4, 8], 1,

                                    p=[0.15, 0.7, 0.1, 0.05])

            time.sleep(0.25*rest[0])
```

```
except KeyboardInterrupt:
```

```
    exit()
```

```
# call main
```

```
if __name__ == '__main__':
```

```
    main()
```
